Week 6 — Functions & Scope

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- Defining functions: parameters and return values
- Default parameter values
- Variable scope: local vs. global
- Refactoring scripts with functions

Apply — ~30 min

- Code-along: greet function with defaults
- Code-along: celsius/fahrenheit converter
- Code-along: refactoring the Number Cruncher

The Problem Functions Solve

Copy-pasting code to reuse it works — until you need to change the logic. Then you have to change it everywhere, and one missed spot is a bug.

Functions let you **write logic once** and call it from anywhere.

Defining a Function

Use `def` to name a block of code. Use `return` to send a value back.

```
def greet(name):
    message = f"Hello, {name}!"
    return message

result = greet("Alex")
print(result)
```

Default Parameter Values

Parameters can have defaults — making them optional.

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Alex"))
print(greet("Alex", "Good morning"))
print(greet("Alex", greeting="Hey"))
```

print vs. return

Two very different things.

	`print()`	`return`
What it does	Displays to the terminal	Sends a value back to the caller
Where the value goes	Nowhere — it's gone	To whoever called the function
Can be used later?	No	Yes — assign to a variable

Check for Understanding #1

What does this code print?

```
def double(n):
    return n * 2

result = double(5)
print(result + 1)
```

A) `10`

B) `11`

C) `None`

D) `TypeError`

Local vs. Global Scope

Variables defined inside a function are **local** — they don't exist outside it.

```
def calculate():
    result = 42
    return result

calculate()
print(result)   # NameError: result is not defined
```

Scope — The Fix

Use `return` to pass the value out.

```
def calculate():
    result = 42
    return result

output = calculate()
print(output)    # 42
```

Check for Understanding #2

Why does this code fail?

```
def set_name():
    name = "Alex"

set_name()
print(name)
```

A) `def` is not the right keyword

B) `name` is a reserved word

C) `name` is local to `set_name` — it doesn't exist outside

D) You need `return name` and `name = set_name()`

Refactoring: Before

A script with all logic at the top level is hard to read and reuse.

```
numbers = [42, 14, 78, 5, 61]

minimum = numbers[0]
for n in numbers:
    if n < minimum:
        minimum = n
print(f"Min: {minimum}")
```

Refactoring: After

Give the logic a name, a parameter, and a return value.

```
def find_min(numbers):
    minimum = numbers[0]
    for n in numbers:
        if n < minimum:
            minimum = n
    return minimum

result = find_min([42, 14, 78, 5, 61])
print(f"Min: {result}")
```

Check for Understanding #3

What is the main benefit of wrapping logic in a function?

A) It makes the code run faster
B) It lets you reuse the logic with different inputs without copy-pasting
C) It prevents all bugs
D) It automatically adds error handling

Time to Apply

Let's refactor — and write some new functions too.

Mentor 2 takes over

Assignment 6 Overview

Part 1 — Four warmup scripts

- `warmup1.py` — `greet(name, greeting="Hello")` with three call styles
- `warmup2.py` — `celsius_to_fahrenheit()` and `fahrenheit_to_celsius()`
- `warmup3.py` — demonstrate scope: NameError, then the fix with return
- `warmup4.py` — `is_valid_score(score)` returns True/False

Part 2 — Refactor the Number Cruncher

- Take Week 5's mini-project and break it into named functions
- `find_min`, `find_max`, `search`, `bubble_sort`, `show_menu`, `main`
- No logic outside a function (except the `numbers` list and `main()` call)

Code-Along 1: greet with Defaults

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Alex"))
print(greet("Alex", "Good morning"))
print(greet("Alex", greeting="Hey"))
```

Code-Along 2: Temperature Converter

Two functions — each with one job.

```
def celsius_to_fahrenheit(c):
    return round((c * 9 / 5) + 32, 1)

def fahrenheit_to_celsius(f):
    return round((f - 32) * 5 / 9, 1)

print(f"0°C = {celsius_to_fahrenheit(0)}°F")
print(f"100°C = {celsius_to_fahrenheit(100)}°F")
print(f"72°F = {fahrenheit_to_celsius(72)}°C")
```

Code-Along 3: Refactored find_min

Show how a top-level script becomes a function.

```
def find_min(numbers):
    minimum = numbers[0]
    for n in numbers:
        if n < minimum:
            minimum = n
    return minimum

def main():
    nums = [42, 14, 78, 5, 61, 29]
    print(f"Min: {find_min(nums)}")

main()
```

Extension Challenge

Ungraded extension: Add a `show_stats(numbers)` function to your refactored Number Cruncher that prints minimum, maximum, and average in one call.

Can you make it reuse `find_min` and `find_max` rather than repeating the logic?

Wrap-Up

Today you learned:

- Defining functions with parameters and return values
- Default parameter values
- Local vs. global scope — and why it matters
- Refactoring scripts into named, reusable functions

This week: Complete warmups 1–4 and the refactored Number Cruncher. Submit via pull request.

Next week: Reading and writing files — your programs will finally work with data that lives outside the code.